



Pulse Guard AI

Intelligent Observability & Event Watchdog

Graduate Vibe Coding Challenge — Project 3 (SRE)

Architect-directed. AI-engineered. Human-in-the-loop.

The Problem

Modern cloud platforms emit **millions of log lines**.

Buried inside are the
error spikes that precede outages.

-  Manual log-watching doesn't scale
-  Static thresholds are noisy & miss creeping degradation
-  Alerts arrive **after** customers already feel the pain

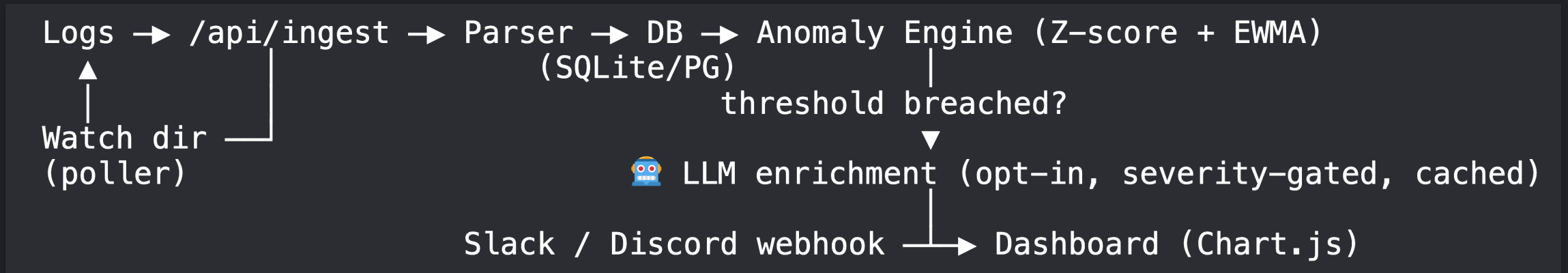
“ SREs need an **automated watchdog** that learns normal and flags abnormal.

The Vision

An **API-first, AI-driven** service that:

1. **Ingests** application/platform logs (any format)
2. **Learns** each service's normal error baseline
3. **Detects** anomalies / spikes with explainable statistics
4. **Explains** spikes with optional LLM root-cause + remediation
5. **Alerts** via real Slack / Discord webhooks
6. **Visualizes** health trends on a live dashboard

Architecture



**API-first · Free DB · Explainable AI · Optional GenAI ·
Real alerts**

Tech Stack

Layer	Technology	Why
API	FastAPI + Uvicorn	Async, auto OpenAPI docs
Database	SQLite / Postgres	Free-tier, one-env-var swap
AI Logic	Rolling Z-score + EWMA	Explainable, dependency-light
GenAI	Opt-in LLM enrichment	Root-cause, label, triage

AI Logic — How Detection Works

Two layers: a **deterministic** detector (always on) + **optional GenAI**.

For each service, per 60-second bucket of **error** events:

- **EWMA baseline** adapts to traffic: $ewma = \alpha \cdot count + (1 - \alpha) \cdot ewma$
- **Rolling Z-score**: $z = (count - mean) / std$
- **Spike** flagged when $count \geq 5$ **and** $z \geq 3$ **and** $count > baseline$
- **Severity**: $z \geq 6 \rightarrow$  critical, $z \geq 3 \rightarrow$  warning

Anomaly Coverage (Enterprise Test Data)

10-service, ~13k-line synthetic dataset — **8 fault patterns + 2 healthy:**

Pattern	Example	Result
Sudden spike	payment-svc	
Gradual creep	search-svc	
Sustained outage	db-proxy	
Cascade	gateway → auth	

Continuous Watchdog Mode

Background **APScheduler** tails `*.log` files in a watch directory:

- ⚡ Ingests only **new** bytes each tick (offset tracking)
- 🔄 Handles **log rotation** / truncation
- 🧠 Runs detection + fires alerts automatically
- 🕒 Controllable via API & dashboard (start / stop / poll now)

```
PULSE_POLL_ENABLED=true uvicorn app.main:app  
echo "... ERROR [svc] boom" >> ingest_watch/svc.log    # auto-detected
```


Real Alerting — Slack & Discord

Auto-detects the webhook flavour from its URL:

- **Slack** → Block Kit incident message
- **Discord** → color-coded rich embed
- **Generic** → raw incident JSON
- **No URL** → local sink (fully demoable offline)

```
PULSE_WEBHOOK_URL=https://hooks.slack.com/services/T/B/X \
uvicorn app.main:app
```



Optional LLM Enrichment (opt-in)

The statistical engine triggers; the LLM **annotates** — never disrupts the deterministic core. Off by default, zero new deps (`httpx`), fail-safe.

-  **Summarizer** → root-cause hypothesis + remediation
-  **Classifier** → short label (e.g. `DB timeout`)
-  **Triage** → business-impact rating (low → critical)

Live Dashboard

-  **Health score** + error-rate trend chart (Chart.js)
-  Detected anomalies table (severity, z-score, baseline, **AI Insight**)
-  Fired webhook alerts log
-  Poller controls + one-click demo / enterprise data
-  AI-enrichment status badge (model + cache hits)
-  Auto-refresh every 15s

Quality & Testing

-  **33 automated tests** (pytest) — parser, detection, scenarios, scheduler
tailing/rotation, Slack/Discord payloads, LLM
enrichment (mocked)
-  True-positive **and** true-negative anomaly coverage
-  LLM tests: severity gating, signature caching, fail-safe, default-off
-  **CI** on push/PR — SQLite + Postgres suites, Docker
build, secret-gated
real-LLM smoke test

Vibe Coding Workflow

-  **Architect** sets vision & rules; **AI** writes 100% of code
-  `prompts.md` — full human-in-the-loop audit log
-  MVP delivered well within the 4–6h target window
-  Bugs fixed by describing them to the AI — **zero manual edits**

Roadmap

-  **Shipped:** opt-in LLM enrichment, Docker + Postgres, CI pipeline
-  Kafka streaming ingestion (opt-in adapter, partition-by-service scale)
-  ML detectors (seasonal Holt-Winters, isolation forest)
-  Native cloud log sources (CloudWatch, Azure Monitor)
-  Per-metric detection (latency, throughput, saturation)



Pulse Guard AI

Detect the spike. Explain it. Trigger the alert.

Thank you.

API-first · Explainable AI + Optional GenAI · Human-in-the-loop